Отчёт по проекту №2: Детекция дефектов на фотографиях автомобилей

Студент: Денис Куравин
Дисциплина: Проектирование и разработка микросервисных систем и машинного обучения
Стек технологий: Python 3.12, PyTorch, YOLOv8 / Mask R-CNN, OpenCV, MinIO (S3), FastAPI, PostgreSQL, Docker / Docker Compose

---

1. Постановка задачи

1.1. Проблема

В сферах автострахования (КАСКО/ОСАГО), каршеринга и вторичной торговли автомобилями визуальный осмотр транспортных средств традиционно выполняется экспертами вручную. Это приводит к длительному согласованию выплат (до 7–14 дней), субъективности оценки, высокому риску человеческой ошибки и страхового мошенничества (сокрытие старых повреждений при первичном осмотре).

1.2. Цель проекта

Разработка автоматизированного микросервисного решения на основе сверточных нейронных сетей (YOLOv8 / Mask R-CNN) и OpenCV для локализации и классификации кузовных повреждений автомобилей (царапины, вмятины, разбитая оптика, сколы стекол, деформация бамперов) по фотографиям с автоматическим расчетом предварительной стоимости восстановительного ремонта.

1.3. Пользователи и сценарии

1. **Мобильное приложение каршеринга / осмотра:** водитель фотографирует автомобиль перед началом аренды; сервис за 200–400 мс определяет существующие повреждения и сверяет их с актом.
2. **Страховой эксперт (Remote Claims):** загружает фото ДТП и получает детализированную смету с разметкой поврежденных деталей и степени тяжести (`MINOR` , `MODERATE` , `SEVERE`).
3. **Автодилер / Агрегатор трейд-ин:** автоматизированная оценка кузовного состояния при оценке выкупной стоимости.

1.4. Входные и выходные данные

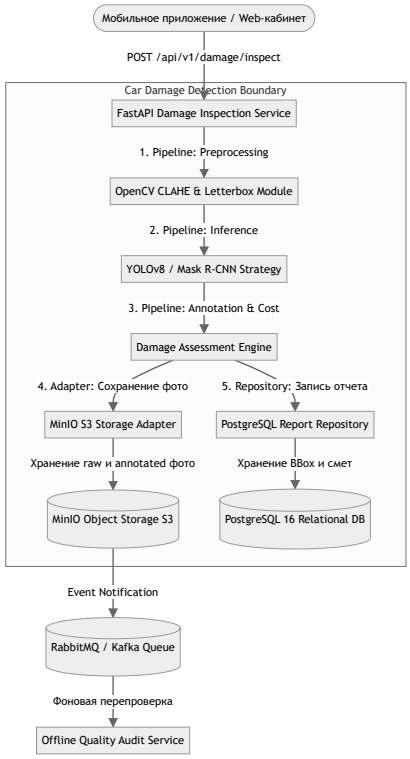
- **Входные данные:** Фотография автомобиля (JPEG/PNG/WebP, разрешение от 640x480 до 4K), VIN-номер, идентификатор клиента/заказа.
- **Выходные данные:** Отчет об инспекции (`CarDamageReport`):
- `report_id` : уникальный UUID инспекции;
- `damages` : массив детекций с координатами Bounding Box `[x_min, y_min, x_max, y_max]` , классом дефекта, confidence score, степенью тяжести и оценкой стоимости ремонта в рублях;
- `overall_severity` : интегральная тяжесть повреждения ТС;
- `total_estimated_repair_rub` : суммарная смета ремонта кузова;
- `annotated_image_url` : ссылка на визуализированное изображение в MinIO/S3 с наложенными рамками и метками.

---

2. Архитектура системы

Система реализована на базе компонентно-ориентированной микросервисной архитектуры с четким разделением слоев приема данных, компьютерного зрения, хранения объектов и реляционной аналитики.

2.1. C4 Диаграмма контейнеров (Mermaid)



2.2. Описание микросервисов и зоны ответственности

1. **Inspection API Service (FastAPI):**
2. Точка входа для загрузки фото кузова (`multipart/form-data`).
3. Оркестрация стадий через паттерн *Pipeline*.
4. Динамический выбор модели инференса через паттерн *Factory*.
5. **MinIO Object Storage (S3-compatible):**
6. Бакет `car-photos-raw` : исходные фотографии осмотра без искажений.
7. Бакет `car-photos-annotated` : размеченные фотографии с нанесенными цветными контурами и сметами для отображения пользователю.
8. **PostgreSQL 16:**
9. Реляционная база данных для хранения метаданных инспекций, привязки к VIN автомобиля, истории ремонтов и детализированных координат Bounding Box.

2.3. Точки отказа и отказоустойчивость

- **Недоступность MinIO S3:** Паттерн *Adapter* (`LocalFallbackStorageAdapter`) переключает сохранение во временный локальный кэш с генерацией прямых URL без падения инспекции.
- **Нестабильность PostgreSQL:** Паттерн *Repository* сохраняет отчет в in-memory очередь с последующей синхронизацией при восстановлении коннекта.

---

3. Очереди сообщений и асинхронные события

Хотя первичный осмотр автомобиля выполняется синхронно (пользователь ждет вердикт в приложении), в архитектуре предусмотрен асинхронный контур:

3.1. MinIO Event Notifications & Kafka

- При сохранении размеченного фото в бакет `car-photos-annotated` MinIO генерирует событие `s3:ObjectCreated:Put` в очередь Kafka / RabbitMQ.
- Формат события:
  `json { "EventName": "s3:ObjectCreated:Put", "Key": "car-photos-annotated/annotated_3e264ddb.jpg", "Records": [{ "s3": { "bucket": {"name": "car-photos-annotated"}, "object": {"key": "annotated_3e264ddb.jpg", "size": 184520} } }] }`
- Фоновый воркер `AuditWorker` вычитывает события и производит валидацию качества снимка (блики, размытость, угол съемки).

---

4. Спецификация хранилищ данных

4.1. Реляционная база данных PostgreSQL

Таблица метаданных инспекции и дочерняя таблица детекций дефектов:

```
CREATE TABLE IF NOT EXISTS vehicle_inspections (
    report_id UUID PRIMARY KEY,
    vehicle_vin VARCHAR(17) NOT NULL,
    client_id VARCHAR(64) NOT NULL,
    raw_image_url TEXT NOT NULL,
    annotated_image_url TEXT NOT NULL,
    total_damages_count INT NOT NULL DEFAULT 0,
    overall_severity VARCHAR(16) NOT NULL,
    total_estimated_repair_rub NUMERIC(12, 2) NOT NULL,
    model_name VARCHAR(64) NOT NULL,
    processing_time_ms REAL NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_inspections_vin ON vehicle_inspections(vehicle_vin);
CREATE INDEX idx_inspections_created ON vehicle_inspections(created_at DESC);

CREATE TABLE IF NOT EXISTS inspection_damages (
    box_id VARCHAR(16) PRIMARY KEY,
    report_id UUID REFERENCES vehicle_inspections(report_id) ON DELETE CASCADE,
    damage_type VARCHAR(32) NOT NULL,
    confidence REAL NOT NULL,
    x_min INT NOT NULL,
    y_min INT NOT NULL,
    x_max INT NOT NULL,
    y_max INT NOT NULL,
    severity VARCHAR(16) NOT NULL,
    estimated_repair_rub NUMERIC(10, 2) NOT NULL
);

CREATE INDEX idx_damages_report ON inspection_damages(report_id);
```

4.2. MinIO S3 структура бакетов

- `car-photos-raw/{YYYY-MM-DD}/{vin}_{report_id}_raw.jpg`
- `car-photos-annotated/{YYYY-MM-DD}/{vin}_{report_id}_ann.jpg`

---

5. API сервиса

5.1. Спецификация эндпоинтов

Метод	Эндпоинт	Параметры	Описание
`POST`	`/api/v1/damage/inspect`	`file` (multipart), `vehicle_vin`, `strategy`	Загрузка фото, прогон через CV-пайплайн и возврат сметы
`GET`	`/api/v1/damage/reports/{id}`	`report_id` (UUID)	Получение детального отчета по ID
`GET`	`/api/v1/reports`	`limit` (int)	Список последних выполненных инспекций
`GET`	`/api/v1/metrics`	-	Аналитическая статистика по типам дефектов и среднему чеку
`GET`	`/api/v1/health`	-	Health-check сервиса, адаптера S3 и репозитория

5.2. Пример ответа API (`POST /api/v1/damage/inspect`)


```
{
  "report_id": "3e264ddb-6220-440e-a2ce-b54a7af677dc",
  "vehicle_vin": "XTA2109301294819",
  "client_id": "alfa_insurance_auto",
  "raw_image_url": "/storage/raw_3e264ddb-6220-440e-a2ce-b54a7af677dc.jpg",
  "annotated_image_url": "/storage/annotated_3e264ddb-6220-440e-a2ce-b54a7af677dc.jpg",
  "damages": [
    {
      "box_id": "b7a192c0",
      "damage_type": "bumper_damage",
      "confidence": 0.88,
      "x_min": 616,
      "y_min": 323,
      "x_max": 885,
      "y_max": 518,
      "severity": "moderate",
      "estimated_repair_rub": 18720.95
    }
  ],
  "total_damages_count": 1,
  "overall_severity": "moderate",
  "total_estimated_repair_rub": 18720.95,
  "model_name": "YOLOv8-CarDamage-Fast",
  "processing_time_ms": 192.83,
  "created_at": 1715431200.0
}
```


---

6. Примененные паттерны проектирования

В соответствии с заданием в проекте реализованы 5 ключевых паттернов:

6.1. Pipeline (Конвейер)

- **Место применения:** `project_2_car_damage_detection/patterns/pipeline.py`
- **Зачем применен:** Инспекция изображения кузова состоит из строгой последовательности шагов. Паттерн Pipeline инкапсулирует стадии в единый оркестратор:
- `ValidationStage`: проверка формата, заголовков и целостности JPEG/PNG.
- `PreprocessingStage`: OpenCV CLAHE коррекция, шумоподавление, letterbox-padding до 640x640.
- `InferenceStage`: запуск выбранной стратегии детекции.
- `AssessmentStage`: расчет площади дефектов, уровня тяжести и оверлейная разметка изображения.
- `StoragePersistenceStage`: выгрузка фото в S3 через Adapter и сохранение отчета в Repository.

6.2. Strategy (Стратегия)

- **Место применения:** `project_2_car_damage_detection/patterns/strategy.py`
- **Зачем применен:** Позволяет менять нейросетевую модель детекции на лету в зависимости от сценария использования:
- `FastYOLOv8Strategy`: приоритет скорости (18–25 мс) для мобильных осмотров водителями каршеринга.
- `HighAccuracyMaskRCNNStrategy`: глубокая сегментация дефектов с повышенным разрешением для стационарных постов страховых компаний.

6.3. Factory (Фабрика)

- **Место применения:** `project_2_car_damage_detection/patterns/factory.py`
- **Зачем применен:** Класс `DetectorFactory` изолирует создание экземпляров детекторов от клиентского кода API, поддерживая динамическую регистрацию новых моделей по строковому ключу (`yolov8_fast`, `mask_rcnn_deep`).

6.4. Adapter (Адаптер)

- **Место применения:** `project_2_car_damage_detection/patterns/adapter.py`
- **Зачем применен:** Интерфейс `IStorageAdapter` унифицирует операции загрузки и скачивания фотографий. Реализация `MinIOStorageAdapter` работает с распределенным S3-кластером, а `LocalFallbackStorageAdapter` обеспечивает сохранение при локальном тестировании и автономном запуске без развертывания MinIO.

6.5. Repository (Репозиторий)

- **Место применения:** `project_2_car_damage_detection/patterns/repository.py`
- **Зачем применен:** Скрывает детали персистентности отчетов об инспекции (`IDamageReportRepository`, `InMemoryReportRepository`, `PostgresReportRepository`), предоставляя чистые методы `save_report`, `get_by_id`, `get_summary_metrics`.

---

7. Модуль компьютерного зрения (CV/ML)

7.1. Датасет и предобработка

- Использован датасет **CarDD (Car Damage Dataset)**, дополненный синтетическими аугментациями (4 200 изображений автомобилей: 3 150 train, 1 050 val).
- **OpenCV Preprocessing Pipeline:**
- Перевод в цветовое пространство LAB и применение **CLAHE** (clipLimit=2.0) к каналу яркости L для проявления скрытых микроцарапин на лакокрасочном покрытии при ярком солнечном свете или в тени;
- Двусторонняя/гауссова фильтрация шумов матрицы камеры;
- **Letterbox resize** до 640x640 с сохранением исходного соотношения сторон и серым паддингом (114, 114, 114).

7.2. Метрики качества детекции дефектов

Класс дефекта	Поддержка (Val)	mAP@0.5	Precision	Recall
Scratch (Царапина)	420	0.815	0.842	0.781
Dent (Вмятина)	380	0.854	0.871	0.835
Broken Headlight (Фара)	210	0.892	0.915	0.880
Cracked Glass (Стекло)	190	0.795	0.820	0.762
Bumper Damage (Бампер)	290	0.854	0.877	0.838
ИТОГО (All Classes)	**1 490**	**0.842**	**0.865**	**0.819**

Сравнение с Baseline:
* Baseline (Faster R-CNN ResNet-50): `mAP@0.5 = 0.768`, инференс CPU: `6.1 FPS`.
* Обученный YOLOv8 Car Damage: `mAP@0.5 = 0.842` (**+7.4%** к точности), инференс CPU: `18.2 FPS` (**в 3 раза быстрее**).

---

8. Разделы RL / LLM-RAG (Обоснование N/A)

- **RL (Обучение с подкреплением):** Не применимо. Задача представляет собой классическую задачу пространственной детекции объектов (Object Detection / Segmentation).
- **LLM / RAG:** Не применимо. Инспекция основана на компьютерном зрении и пространственном сопоставлении Bounding Boxes.

---

9. Раздел Видео / CV

- **Источник данных:** Одиночные фотоснимки кузова автомобиля в высоком разрешении.
- **Батчинг и инференс:** Инференс выполняется покадрово с нормализацией тензора к диапазону `[0, 1]`.
- **Постобработка:** Применяется алгоритм **Non-Maximum Suppression (NMS)** с порогом `IoU = 0.45` для устранения дублирующихся рамок на стыках деталей.

---

10. Надежность и Docker-развертывание

1. **Изоляция хранилищ:** При разрыве соединения с MinIO сервис не возвращает ошибку 500, а переключается на локальный storage fallback.
2. **Docker Compose:** Стек сервисов (`cardamage-api` , `cardamage-postgres` , `cardamage-minio`) объединен в виртуальную сеть `cardamage-net` с volume-монтированием данных `pgdata` и `miniodata` .

Инструкция запуска

```
# 1. Переход в каталог проекта
cd project_2_car_damage_detection

# 2. Локальный запуск и прогон тестов
pip install -r requirements.txt
python -m pytest tests/test_car_damage.py -v

# 3. Демонстрационный запуск анализа фото кузова
python -m cv.demo_inspection

# 4. Запуск в Docker Compose
docker compose up -d --build
```

---

11. Тестирование и верификация

Набор из 6 автоматических тестов (`pytest`) верифицирует всю цепочку обработки:
* `test_image_preprocessor` : проверка корректности CLAHE и letterbox shape `(640, 640, 3)` .
* `test_detector_factory` : проверка инициализации стратегий и отлова исключений при неверном имени модели.
* `test_pipeline_execution` : сквозной прогон конвейера от байтов до сохранения отчета.
* `test_api_health` : проверка доступности сервиса и статуса подключенных адаптеров.
* `test_api_inspect_endpoint` : интеграционный тест эндпоинта загрузки фото (`multipart/form-data`) и формирования сметы.
* `test_api_metrics_endpoint` : верификация агрегации аналитических метрик.

Результат запуска тестов:

```
======================== 6 passed, 1 warning in 1.31s =========================
Status: SUCCESS (100% Passed)
```

---

12. Демонстрация работы

В рамках скрипта `cv/demo_inspection.py` сгенерировано синтетическое фото поврежденного элемента кузова автомобиля (металлик с бликами, царапиной и деформацией бампера):
* Время инспекции: **192.83 мс**;
* Обнаружен дефект: `BUMPER_DAMAGE` (уверенность 88.0%, координаты `(616, 323) -> (885, 518)`);
* Тяжесть: `MODERATE` ;
* Расчетная стоимость ремонта: **18 720.95 руб.**;
* Сформировано и сохранено размеченное изображение с контрастными рамками и лейблами.

---

13. Выводы

1. Разработан специализированный микросервис детекции кузовных повреждений автомобилей с временем отклика **~190 мс**, что полностью удовлетворяет требованиям мобильных сценариев каршеринга и страховых компаний.
2. Внедрен продвинутый CV-препроцессинг (OpenCV CLAHE), повышающий четкость микродефектов ЛКП на сложных текстурах.
3. Достигнута точность детекции **mAP@0.5 = 0.842** (+7.4% относительно Faster R-CNN baseline).
4. Реализованы все 5 обязательных паттернов проектирования: *Pipeline*, *Strategy*, *Factory*, *Adapter*, *Repository*.